

Priority Based Schedule Conflict Resolver and Free Slot Recommender Using C++

Rifat Adly Pratama, Zia Maharani Putri A., and Valerie Graceline Darsojo

Group 4, Programming IUP1

Faculty of Mathematics and Natural Sciences, Universitas Gadjah Mada, Yogyakarta, Indonesia

Abstract— University students routinely manage lectures, group meetings, organization activities, and personal commitments within a limited weekly time budget, and overlapping activities force immediate decisions about what to keep and what to move. Standard calendar applications often work in passive mode, without the automated decision-making logic when schedule collisions occur. This paper presents a console-based C++ schedule manager for a single user that detects overlapping activities across a seven-day week, applies a transparent, user-defined priority rule, and recommends a maximum of five feasible alternative time slots for displaced flexible activities. Activities are stored with a title, day, start and end time, a priority from 1 to 5, and a fixed or flexible type. Conflict handling follows a priority-based greedy heuristic: fixed activities are always retained, the higher-priority flexible activity is retained otherwise, ties are referred to the user, and overlaps of 10 minutes or less trigger an ignore-or-relocate prompt. The system recommends options but never reschedules automatically, and schedules are persisted with fstream. The implementation provides rule based conflict handling and free slot recommendations to reduce manual scheduling effort.

Keywords— *Time poverty, Schedule, Timeline*

I. INTRODUCTION

In higher education, university students routinely manage complex and dynamic personal schedules comprising academic lectures, group assignment deadlines, and extracurricular activities. As schedule density increases, individuals frequently encounter time collisions—situations where two or more mandatory tasks overlap within the same temporal interval. Behavioral psychology research indicates that managing these continuous temporal conflicts induces a condition known as "Time Poverty" and significantly increases "Cognitive Load". The constant mental demand of evaluating, cancelling, and manually rescheduling tasks exhausts the human decision-making capacity

a) Limitation of existing tools & gap analysis

Every personal information management tools—such as digital calendar applications and automated notification software—are widely used their operational capabilities remain predominantly passive. These systems record events and trigger static reminders; however, when schedule overlaps occur, they fail to provide decision-making logic or algorithmic resolution strategies. Consequently, the mental burden of evaluating task criticalities, identifying alternative open hours, and resolving schedule collisions is left entirely to the user.

II. PROBLEM FORMULATION AND SCOPE

The System is a console-based schedule manager for one student. Each activity a is described by a title, a start time $s(a)$ and end time $e(a)$ in 24-hour format, a priority $p(a) \in \{1, \dots, 5\}$ where 5 is the highest, and a type that is either fixed or flexible. A fixed activity cannot be moved by the system; a flexible activity may receive alternative slot recommendations. The scope of the system is summarized in Table 1.

TABLE I.
PROJECT SCOPE AND ASSUMPTIONS

Aspect	Scope
Target user	Students and other individuals managing overlapping activities
Schedule range	Seven day weekly schedule
Time format	24-hour
Priority	1 = lowest, 5 = highest
Conflict handling	Recommends options; no automatic rescheduling
Conflict tolerance	Overlaps ≤ 10 min trigger an ignore-or-relocate prompt
Output	Final schedule, detected conflicts, recommended free slots
Storage	Text file via fstream

The goal is, given a new activity and the current schedule, to decide whether the activity can be added as-is, which of two conflicting activities is resolved, and where a flexible activity could be placed instead. The user remains responsible for the final decision.

III. METHODOLOGY

A. Collision Detection

Two activities on the same day, A and B, overlap when $(Start_A < End_B) \wedge (End_A > Start_B)$.

Because the comparisons are strict, activities that merely touch (one ends exactly when the other starts) do not conflict. For example, an Algorithms Class from 08:00 to 10:00 and a Group Meeting from 09:00 to 10:30 on Monday conflict, whereas a Student Club Activity from 08:00 to 10:00 on Tuesday does not, since it falls on a different day. The activity list is sorted by start time so that each new activity is checked against the relevant scheduled activities.

B. Conflict Tolerance

Real schedules often contain small overlaps, such as a briefing that begins a few minutes before the previous session ends. The overlap duration δ , in minutes, is defined as

$$\delta = \min(End_A, End_B) - \max(Start_A, Start_B).$$

If $0 < \delta \leq 10$ minutes, the conflict is classed as minor and the system asks the user to either ignore the overlap or relocate an activity. If δ exceeds 10 minutes, it is major and the conflict resolution rules below are applied. For instance, an Algorithms Class from 07:00 to 08:00 and a Committee Briefing from 07:55 to 09:00 produce a 5-minute minor conflict and the prompt "Ignore or Relocate?".

C. Conflict Resolution Rules

When a new activity conflicts with an existing one, the following rules are applied in order.

- 1) *Fixed versus flexible*: the fixed activity is kept.
- 2) *Flexible versus flexible*: the activity with the higher priority is kept in its current slot.
- 3) *Equal priority*: the system does not choose automatically and asks the user to decide.
- 4) *Fixed versus fixed*: both activities are kept, and the system warns the user about the conflict without automatically changing either activity.
- 5) *Displacement and relocation*: the lower-priority flexible activity is flagged for rescheduling, and the system searches for free slots of sufficient duration for it.

D. Free Slot Recommendation

A free slot is an empty interval that (i) does not overlap any scheduled activity, (ii) is at least as long as the displaced activity, (iii) lies within the schedule boundary (for example 07:00–22:00), and (iv) can be selected by the user as the alternative. Given the sorted activities of a day, the gaps between consecutive activities, and between the boundaries and the first and last activity, are the candidate intervals. A gap that starts at time s and ends at time e is valid for a displaced activity of duration d when

$$e - s \geq d.$$

Table II illustrates the rule. A 90-minute Group Meeting can move to 15:00–16:30 on Monday, after a 12:00–15:00 activity, but cannot move into an 18:00–19:00 gap on the same day because the gap is only 60 minutes long. At most five valid slots are recommended; if none exists, the system informs the user rather than forcing an incorrect reschedule.

TABLE II
FREE-SLOT VALIDITY EXAMPLES

Day	Existing	To move	Result
Mon	12:00–15:00 activity	Group Meeting, 90 min	Valid: 15:00–16:30
Mon	Free 18:00–19:00	Group Meeting, 90 min	Not valid: too short

IV. SYSTEM DESIGN AND IMPLEMENTATION

A. C++ Components

The program is implemented with standard C++ facilities: a *struct* holds each activity's data; *std::vector* stores the dynamic list of activities; *std::sort* orders activities by start time; dedicated functions perform time validation, overlap detection, priority comparison, and free-slot search; and *std::fstream* saves and loads the schedule from a .txt file.

B. System Workflow

The workflow, shown as a flowchart in the project presentation, proceeds as follows. (1) The user enters the activity title, day, start and end times, priority, and type. (2) The input is validated: the start time must be earlier than the end time, otherwise the user re-enters it. (3) The system scans for overlaps using (1). (4) With no conflict, the activity is added. For a minor conflict (≤ 10 min), the tolerance prompt is shown. For a major conflict (> 10 min), the resolution rules are applied. (5) If an activity is displaced,

maximum of five free slots are recommended, and the user selects one or cancels. (6) The finalized schedule is written to a local file.

C. Usage Example and Simulation

A simulation of our C++ program was conducted with the user entering three schedules. Among these schedules, some of them overlap. The program generates output based on the flexibility of the schedules.

Step	Action	System Response
1	Add Calculus I, Monday 07.15 - 08.55, Fixed	Activity Added
2	Add Tennis, Monday 08.30 - 10.15, Flexible, Priority 3	Conflict detected
3	System detects Fixed vs Flexible	Calculus I is kept
4	Free slots displayed	Several alternatives shown
5	User selects Monday 14.10-15.55	Tennis is rescheduled
6	Schedule saved	Both activities are stored
7	Add Statistical Methods, Monday 08.45 - 09.55, Fixed	Conflict detected
8	System detects overlap between two fixed schedules	Warning message is shown
9	Schedule saved	Activities are stored
10	Show schedule	System shows the current schedule created

Console:

```
Choose: 1
Enter activity name: Calculus 1
Enter day (1=Monday ... 7=Sunday): 1
Enter start time (HH MM): 07 15
Enter end time (HH MM): 08 55
Select type:
1. Fixed
2. Flexible
Choose: 1
Activity added successfully.

===== CLASS SCHEDULE MANAGER =====
1. Add Schedule
2. Show Schedule
3. Delete Schedule
4. Exit
Choose: 1
Enter activity name: Tennis
Enter day (1=Monday ... 7=Sunday): 1
Enter start time (HH MM): 08 30
Enter end time (HH MM): 10 15
Select type:
1. Fixed
2. Flexible
Choose: 2
Enter priority (1-5): 3

===== CONFLICT =====
New activity      : Tennis
Existing activity : Calculus 1
Overlap          : 25 minutes

Existing Fixed activity stays.
New Flexible activity must be rescheduled.
```

```

Recommended free slots:
1. Monday 08:55 - 10:40
2. Monday 10:40 - 12:25
3. Monday 12:25 - 14:10
4. Monday 14:10 - 15:55
5. Monday 15:55 - 17:40
6. Manual input
0. Cancel
Choose: 4

Activity added successfully.

===== CLASS SCHEDULE MANAGER =====
1. Add Schedule
2. Show Schedule
3. Delete Schedule
4. Exit
Choose: 1

Enter activity name: Statistical Methods
Enter day (1=Monday ... 7=Sunday): 1
Enter start time (HH MM): 08 45
Enter end time (HH MM): 09 55
Select type:
1. Fixed
2. Flexible
Choose: 1

===== CONFLICT =====
New activity      : Statistical Methods
Existing activity : Calculus 1
Overlap           : 10 minutes
[WARNING] Both activities are Fixed.
Please check the schedule again. No activity was changed.

Activity added successfully.

```

```

===== CLASS SCHEDULE MANAGER =====
1. Add Schedule
2. Show Schedule
3. Delete Schedule
4. Exit
Choose: 2

===== SCHEDULE =====
1. Calculus 1
   Day      : Monday
   Time     : 07:15 - 08:55
   Type     : Fixed
2. Statistical Methods
   Day      : Monday
   Time     : 08:45 - 09:55
   Type     : Fixed
3. Tennis
   Day      : Monday
   Time     : 14:10 - 15:55
   Type     : Flexible
   Priority : 3
=====

```

V. DISCUSSION

Currently, this program still has several limitations as our program is eventually still in development. Some of the limitations are, the system serves one user and does not support group scheduling; priorities are assigned manually; it does not integrate with Google Calendar, Outlook, or other external calendars; recommendations are purely rule-based; and it does not measure time poverty, stress, or cognitive load.

To further optimize our work, in the future we plan to possibly develop the program in several ways such as Planned extensions include weekly and monthly scheduling, repeated activities, travel-time and break-time constraints, multiple user schedules for finding common meeting times, and calendar synchronization. Replacing the greedy rule with dynamic programming for weighted interval scheduling [4] would allow the system to maximize total retained priority, and a graphical or web-based interface would make the tool accessible beyond the console. A user study measuring whether the recommendations reduce scheduling effort would also be needed to evaluate practical benefit, which this work does not claim.

VI. CONCLUSION

This paper presented a console-based C++ schedule manager that detects overlapping activities, classifies them as minor or major using a 10-minute tolerance, resolves major conflicts through a transparent fixed-first, priority-based rule set, and recommends a maximum of five feasible free slots for displaced flexible activities. The design intentionally keeps the user in control and relies on a simple, fast greedy heuristic that is suited to a small implementation. Its main limitation is that choosing the best option at each conflict does not always give the best overall schedule. In the future, the system could handle more constraints and connect to calendar apps.

REFERENCES

- [1] C. Wladis, A. C. Hachey, and K. M. Conway, "It's about time, part II: Does time poverty contribute to inequitable college outcomes by gender and race/ethnicity?," *AERA Open*, vol. 10, Art. no. 23328584241237971, 2024.
- [2] S. Abdipoor, R. Yaakob, S. L. Goh, and S. Abdullah, "Meta-heuristic approaches for the university course timetabling problem," *Intell. Syst. Appl.*, vol. 19, Art. no. 200253, 2023, doi: 10.1016/j.iswa.2023.200253.
- [3] H. Babaei, J. Karimpour, and A. Hadidi, "A survey of approaches for university course timetabling problem," *Comput. Ind. Eng.*, vol. 86, pp. 43–59, 2015, doi: 10.1016/j.cie.2014.11.010.